USB TETHERING ON LINUX SERVER

Complete Setup, Troubleshooting & Advanced Configuration Guide

Covers: Ubuntu · Debian · RHEL · Arch Linux

Android & iPhone Tethering | NetworkManager & systemd-networkd | Persistent Config

Version 1.0 | April 2026

1. Introduction

USB tethering allows a smartphone (Android or iPhone) to share its mobile data connection with a Linux server over a USB cable. Unlike Wi-Fi or Bluetooth tethering, USB tethering is faster, more stable, does not drain the phone battery as quickly (the phone usually charges while connected), and does not require any wireless configuration on the server side.

This is particularly useful for:

- Headless servers with no Wi-Fi card
- Bootstrapping a server that has no network connection yet
- Fallback internet when your primary LAN/WAN is down
- Remote SSH access via Cloudflare Tunnel or similar, without a dedicated router
- Rapid deployment in a new location with just a phone and a USB cable

1.1 How It Works

When USB tethering is enabled on a phone, the phone presents itself to the Linux host as a USB network adapter (a RNDIS/CDC-ECM device on Android, or NCM on iPhone). Linux then sees a new network interface — typically named usb0, enp0s20u1, or similar — and can obtain an IP address via DHCP from the phone's internal DHCP server.

Parameter	Typical Value	Notes
Phone DHCP range	192.168.42.x (Android) 172.20.10.x (iPhone)	Assigned by phone
Server IP	192.168.42.2 (or similar)	Assigned automatically via DHCP
Gateway	192.168.42.129 (Android)	Phone acts as NAT gateway
DNS	Google 8.8.8.8 or carrier DNS	Provided by phone
Interface name	usb0 / enp0s20u1 / eth1	Depends on kernel & USB port

2. Prerequisites

2.1 Hardware Requirements

- A Linux server (Ubuntu 20.04+, Debian 11+, RHEL/Rocky 8+, Arch, or similar)
- An Android smartphone (Android 4.0+) or iPhone (iOS 12+)

- A USB cable (preferably the original data cable — some cables are charge-only)
- USB port on the server (USB 2.0 or 3.0 — both work)

WARNING Always use a cable that supports data transfer. If tethering does not work, try a different cable first — this is a very common issue.

2.2 Software / Kernel Requirements

USB tethering is supported out-of-the-box on most modern Linux kernels (4.x+). The relevant kernel modules are:

Module	Purpose	Phone Type
rndis_host	Remote NDIS — most Android phones	Android
cdc_ether	CDC Ethernet — some Android phones	Android
cdc_ncm	Network Control Model — iPhones	iPhone (iOS)
ipheth	iPhone USB Ethernet — older iOS	iPhone (older)
usbnet	Core USB networking layer	All

Check if modules are available

Kernel Module Check

```
# Check if RNDIS module is available (Android)
modinfo rndis_host

# Check if CDC NCM module is available (iPhone)
modinfo cdc_ncm

# List all loaded modules related to USB networking
lsmod | grep -E 'rndis|cdc|ipheth|usbnet'
```

3. Android USB Tethering Setup

3.1 Enable Tethering on Android Phone

1. Connect your Android phone to the server via USB cable.
2. On your phone, go to: Settings → Network & Internet → Hotspot & Tethering
3. Toggle "USB tethering" to ON.
4. If prompted, select "File Transfer" or "USB Tethering" mode (NOT "Charging Only").

INFO On some Samsung/MIUI devices, the path is: Settings → Connections → Mobile Hotspot and Tethering → USB Tethering

3.2 Verify the Interface Appears on Linux

Detecting the USB Tethering Interface

```
# Right after plugging in and enabling tethering on the phone:

# Method 1: Check dmesg for the new USB device
dmesg | tail -30

# You should see lines like:
# [12345.678] usb 1-1: new high-speed USB device number 3 using xhci_hcd
# [12345.890] rndis_host 1-1:1.0 usb0: register 'rndis_host'

# Method 2: List all network interfaces
ip link show

# Method 3: Look specifically for USB-related interfaces
ip link show | grep -E 'usb|enp.*u'

# Method 4: Use nmcli (NetworkManager)
nmcli device status
```

3.3 Identify the Correct Interface Name

The interface name varies depending on your kernel version and systemd version:

- usb0 — older kernels / older naming scheme
- enp0s20u1 — typical on modern Ubuntu/Debian with predictable naming
- enp0s20u1u1 — if plugged into a USB hub
- eth1 — some minimal/embedded Linux setups

Identify New Interface

```
# Find the interface using its MAC address prefix or by watching changes
# Run BEFORE plugging in:
ip link show > /tmp/before.txt

# Plug in phone and enable tethering, then run:
ip link show > /tmp/after.txt

# Compare the two outputs to see what's new:
diff /tmp/before.txt /tmp/after.txt
```

3.4 Bring Up the Interface and Get IP via DHCP

Using ip + dhclient (manual method)

Manual IP Assignment

```
# Replace 'usb0' with your actual interface name
IFACE=usb0

# Bring the interface up
sudo ip link set $IFACE up

# Request an IP via DHCP from the phone
sudo dhclient $IFACE

# Verify you got an IP
ip addr show $IFACE

# Expected output:
# 3: usb0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_coda state UNKNOWN
#     link/ether xx:xx:xx:xx:xx:xx brd ff:ff:ff:ff:ff:ff
#     inet 192.168.42.167/24 brd 192.168.42.255 scope global dynamic usb0

# Test internet connectivity
ping -c 4 8.8.8.8
curl -I https://google.com
```

Using NetworkManager (recommended for desktops/servers with nmcli)

NetworkManager Method

```
# Check current device status
nmcli device status

# If the device shows 'unmanaged', set it to managed:
nmcli device set usb0 managed yes

# Connect using DHCP (NetworkManager auto-detects)
nmcli device connect usb0

# Verify
nmcli device show usb0

# Check routing
ip route show
```

Using systemd-networkd

systemd-networkd Method

```
# Create a .network file for the USB interface
sudo nano /etc/systemd/network/25-usb-tethering.network

# Paste the following content:
# ----------------------------------------------
# [Match]
# Name=usb*
# Driver=rndis_host
#
# [Network]
# DHCP=yes
# ----------------------------------------------

# Restart networkd
sudo systemctl restart systemd-networkd

# Check status
networkctl status
networkctl list
```

4. iPhone USB Tethering Setup

4.1 Enable Tethering on iPhone

5. Go to Settings → Personal Hotspot on your iPhone.
6. Enable "Allow Others to Join".
7. Connect iPhone to Linux server via USB cable.
8. On iPhone, when prompted, tap "Trust" for the computer.

WARNING iPhone USB tethering requires the ipheth or cdc_ncm kernel module. Most modern Linux kernels include this, but some minimal/server kernels may not.

4.2 Install Required Packages

iPhone Dependencies

```
# Ubuntu / Debian
sudo apt update
sudo apt install -y libimobiledevice-utils usbmuxd ifuse
```

```
# RHEL / Rocky / AlmaLinux / Fedora
sudo dnf install -y libimobiledevice usbmuxd

# Arch Linux
sudo pacman -S libimobiledevice usbmuxd

# Enable usbmuxd service
sudo systemctl enable --now usbmuxd
```

4.3 Pair the iPhone

iPhone Pairing

```
# List connected iOS devices
idevice_id -l

# Pair with the device (tap Trust on iPhone when prompted)
idevicepair pair

# Verify pairing
idevicepair validate

# Expected output: SUCCESS: Paired with device ...
```

4.4 Bring Up the Interface

Bring Up iPhone Interface

```
# After pairing, a network interface should appear
ip link show | grep -E 'eth|enp'

# The iPhone interface is usually eth1 or enp0s20u1
# Enable it and get an IP via DHCP
sudo dhclient eth1     # replace eth1 with your interface

# Verify
ip addr show eth1
ping -c 3 8.8.8.8
```

5. Making USB Tethering Persistent (Auto-Connect)

By default, you need to run dhclient manually every time you plug in the phone. Here is how to make it automatic and persistent across reboots.

5.1 Using udev Rules (Universal Method)

udev rules trigger automatically when a USB device is plugged in. This is the most reliable way to auto-configure tethering.

Find Device IDs

```
# First, find the vendor and product IDs of your phone
# Plug in the phone with tethering enabled, then run:
lsusb

# Example output:
# Bus 002 Device 003: ID 04e8:6864 Samsung Electronics Co., Ltd
#                         ^^^^ ^^^^
#                    vendor_id  product_id

# Another way — watch for USB events:
udevadm monitor --udev --subsystem-match=usb/usb_device
# (plug phone in now and check output)

# Get detailed attributes of the device:
udevadm info -a -p $(udevadm info -q path -n /dev/bus/usb/002/003)
```

/etc/udev/rules.d/90-usb-tethering.rules

```
# Create the udev rule file
sudo nano /etc/udev/rules.d/90-usb-tethering.rules

# ----------------------------------------------
# Paste this content (replace XXXX with your vendor ID):

SUBSYSTEM=="net", ACTION=="add", DRIVERS=="rndis_host", \
    ATTR{address}!="", \
    RUN+="/usr/local/bin/usb-tether-up.sh $name"

# For iPhone, use cdc_ncm or ipheth instead of rndis_host:
SUBSYSTEM=="net", ACTION=="add", DRIVERS=="cdc_ncm", \
    ATTR{address}!="", \
    RUN+="/usr/local/bin/usb-tether-up.sh $name"
# ----------------------------------------------
```

Helper Script + Reload udev

```
# Create the helper script
sudo nano /usr/local/bin/usb-tether-up.sh
```

```
# ----------------------------------------------
#!/bin/bash
IFACE=$1
LOG="/var/log/usb-tether.log"

echo "$(date): USB tether interface $IFACE appeared" >> $LOG

# Bring up interface
ip link set $IFACE up

# Wait for interface to stabilize
sleep 2

# Request IP via DHCP
dhclient -v $IFACE >> $LOG 2>&1

echo "$(date): dhclient done for $IFACE" >> $LOG

# Optional: add a default route via this interface
# ip route add default dev $IFACE metric 200
# ----------------------------------------------

# Make executable
sudo chmod +x /usr/local/bin/usb-tether-up.sh

# Reload udev rules
sudo udevadm control --reload-rules
sudo udevadm trigger
```

5.2 Using NetworkManager (Easiest on Ubuntu)

/etc/NetworkManager/system-connections/USB-Tethering.nmconnection

```
# NetworkManager can be configured to auto-manage USB tether interfaces

# Create a connection profile for USB tethering
sudo nmcli connection add \
  type ethernet \
  con-name 'USB-Tethering' \
  ifname '*' \
  connection.autoconnect yes \
  ipv4.method auto \
  ipv6.method ignore

# Or create the config file directly:
sudo nano /etc/NetworkManager/system-connections/USB-Tethering.nmconnection
```

```
# ----------------------------------------------
[connection]
id=USB-Tethering
type=ethernet
autoconnect=true
autoconnect-priority=10

[ethernet]
wake-on-lan=default

[ipv4]
method=auto

[ipv6]
addr-gen-mode=stable-privacy
method=ignore
# ----------------------------------------------

# Set correct permissions (required!)
sudo chmod 600 /etc/NetworkManager/system-connections/USB-Tethering.nmconnection

# Reload
sudo nmcli connection reload
```

5.3 Using systemd Service for Auto-Reconnect

/etc/systemd/system/usb-tether.service

```
# Create a systemd service that monitors for USB tether
sudo nano /etc/systemd/system/usb-tether.service

# ----------------------------------------------
[Unit]
Description=USB Tethering Auto-Connect
After=network.target
Wants=network.target

[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/usr/local/bin/usb-tether-up.sh usb0
ExecStop=/sbin/ip link set usb0 down
Restart=on-failure

[Install]
WantedBy=multi-user.target
# ----------------------------------------------
```

```
sudo systemctl daemon-reload
sudo systemctl enable usb-tether.service
sudo systemctl start usb-tether.service

# Check status
sudo systemctl status usb-tether.service
```

6. Routing & DNS Configuration

6.1 Check Default Route

Routing Table

```
# View current routing table
ip route show

# Example output with USB tether as default:
# default via 192.168.42.129 dev usb0 proto dhcp src 192.168.42.167
# 192.168.42.0/24 dev usb0 proto kernel scope link src 192.168.42.167

# If no default route exists via USB, add one manually:
sudo ip route add default via 192.168.42.129 dev usb0

# To give USB tether a higher priority (lower metric = higher priority):
sudo ip route add default via 192.168.42.129 dev usb0 metric 100
```

6.2 DNS Configuration

DNS Setup

```
# Check current DNS servers
cat /etc/resolv.conf

# If DNS is missing, add manually:
sudo nano /etc/resolv.conf

# ----------------------------------------------
nameserver 8.8.8.8
nameserver 8.8.4.4
nameserver 1.1.1.1
# ----------------------------------------------

# On Ubuntu with systemd-resolved:
```

```
sudo systemd-resolve --interface=usb0 --set-dns=8.8.8.8

# Or configure resolvconf:
echo 'nameserver 8.8.8.8' | sudo resolvconf -a usb0.dhcp

# Test DNS resolution
dig google.com @8.8.8.8
nslookup google.com
```

6.3 Multiple Interfaces — Policy-Based Routing

If your server has both a LAN connection and USB tethering, you may want to route specific traffic through the phone (e.g., SSH fallback). Here is how to set up policy-based routing:

Policy-Based Routing

```
# Add a separate routing table for USB tether
echo '200 usb_tether' | sudo tee -a /etc/iproute2/rt_tables

# Add routes to the custom table
sudo ip route add default via 192.168.42.129 dev usb0 table usb_tether
sudo ip route add 192.168.42.0/24 dev usb0 scope link table usb_tether

# Add a rule: packets FROM the USB interface use the usb_tether table
sudo ip rule add from 192.168.42.167 table usb_tether priority 100

# Verify
ip rule show
ip route show table usb_tether
```

7. Troubleshooting

7.1 Interface Does Not Appear

Interface Not Found

```
# Step 1: Check if phone is detected at USB level
lsusb

# Step 2: Check kernel messages
dmesg | tail -50 | grep -iE 'usb|rndis|tether|cdc'

# Step 3: Load the RNDIS module manually
sudo modprobe rndis_host
```

```
sudo modprobe cdc_ether

# For iPhone:
sudo modprobe cdc_ncm
sudo modprobe ipheth

# Step 4: Check if module loaded
lsmod | grep rndis

# Step 5: Check USB device details
usb-devices | grep -A 20 'Vendor'
```

WARNING The most common reason is the phone is in 'Charging only' mode. Unlock your phone, swipe down the notification bar, tap the USB notification, and select 'File Transfer' or 'USB Tethering'.

7.2 Interface Appears but No IP Address

No IP Address

```
# Check interface state
ip link show usb0

# If DOWN, bring it up first
sudo ip link set usb0 up

# Try dhclient with verbose output
sudo dhclient -v usb0

# Or use dhcpcd instead
sudo apt install dhcpcd5
sudo dhcpcd usb0

# Check for DHCP errors
sudo journalctl -u dhclient -n 50
sudo journalctl -u NetworkManager -n 50 | grep -i usb

# Check that the phone's hotspot is actually active
# Verify by checking phone screen — hotspot icon should be visible
```

7.3 IP Obtained but No Internet

Internet Not Working

```
# Check routing table
ip route show
```

```
# Ping the phone's gateway directly
ping -c 3 192.168.42.129

# If gateway is reachable but internet is not:
# Check if the default route points to usb0
ip route get 8.8.8.8

# If default route is wrong, fix it:
sudo ip route del default
sudo ip route add default via 192.168.42.129 dev usb0

# Test DNS
ping -c 3 8.8.8.8        # Direct IP — tests routing
ping -c 3 google.com      # Domain name — tests DNS too

# Check if phone has mobile data (obvious but worth checking!)
# Check phone's mobile data toggle
```

7.4 Connection Drops After a While

Connection Drops Fix

```
# Keep-alive: prevent DHCP lease expiry issues
# Add a cron job to renew DHCP periodically
sudo crontab -e

# Add this line (renew every 10 minutes):
*/10 * * * * /sbin/dhclient usb0 >> /var/log/usb-tether.log 2>&1

# Check for USB power management issues
cat /sys/bus/usb/devices/*/power/control

# Disable USB autosuspend for the phone's USB port
# Find the correct USB device path from lsusb first
echo 'on' | sudo tee /sys/bus/usb/devices/1-1/power/control

# Permanent fix via udev:
sudo nano /etc/udev/rules.d/91-usb-power.rules
# Add:
ACTION=="add", SUBSYSTEM=="usb", ATTR{idVendor}=="04e8", \
  ATTR{power/control}="on"
# (replace 04e8 with your phone's vendor ID)
```

7.5 iPhone: 'Trust This Computer' Not Appearing

iPhone Trust Issue

```
# Ensure usbmuxd is running
sudo systemctl status usbmuxd
sudo systemctl start usbmuxd

# Unpair and re-pair
idevicepair unpair

# Unplug iPhone, wait 5 seconds, replug
# Unlock iPhone before plugging
# Then pair again:
idevicepair pair

# If still not working, reset iPhone trust settings:
# iPhone: Settings → General → Transfer or Reset iPhone → Reset → Reset Location
& Privacy

# Check libimobiledevice version
ideviceinfo --version

# Update if needed
sudo apt upgrade libimobiledevice-utils
```

7.6 Diagnostic Commands Reference

Command	What It Tells You
lsusb	Lists all USB devices — confirms phone is detected
dmesg | tail -30	Kernel messages — shows USB events and errors
ip link show	All network interfaces and their state
ip addr show	IP addresses assigned to each interface
ip route show	Routing table — shows how traffic is routed
nmcli device status	NetworkManager device states
dhclient -v usb0	Verbose DHCP — shows lease negotiation details
journalctl -u NetworkManager -f	Live NetworkManager logs
ping -c 3 192.168.42.129	Test connectivity to phone (gateway)
cat /etc/resolv.conf	Current DNS server configuration

8. Advanced Topics

8.1 SSH Access from Outside via Cloudflare Tunnel over USB Tether

Once your server has internet via USB tethering, you can expose SSH anywhere using Cloudflare Tunnel — no port forwarding or static IP required.

Cloudflare Tunnel over USB Tether

```
# Install cloudflared
curl -L https://pkg.cloudflare.com/cloudflare-main.gpg \
  | sudo tee /usr/share/keyrings/cloudflare-main.gpg > /dev/null

echo "deb [signed-by=/usr/share/keyrings/cloudflare-main.gpg] \
  https://pkg.cloudflare.com/cloudflared $(lsb_release -cs) main" \
  | sudo tee /etc/apt/sources.list.d/cloudflared.list

sudo apt update && sudo apt install cloudflared

# Authenticate
cloudflared tunnel login

# Create tunnel
cloudflared tunnel create my-server

# Expose SSH
cloudflared tunnel --url ssh://localhost:22

# Access from your laptop:
# ssh -o ProxyCommand='cloudflared access ssh --hostname YOUR_TUNNEL_URL'
user@localhost
```

8.2 Share USB Tether Internet to LAN (IP Masquerading)

If your server is connected to a LAN and you want to share the phone's internet with other LAN devices:

Share Internet to LAN

```
# Enable IP forwarding
echo 'net.ipv4.ip_forward=1' | sudo tee -a /etc/sysctl.conf
sudo sysctl -p

# Set up NAT (masquerade)
# usb0 = phone-connected interface
# eth0 = LAN interface

sudo iptables -t nat -A POSTROUTING -o usb0 -j MASQUERADE
sudo iptables -A FORWARD -i eth0 -o usb0 -j ACCEPT
```

```
sudo iptables -A FORWARD -i usb0 -o eth0 -m state \
  --state RELATED,ESTABLISHED -j ACCEPT

# Make iptables rules persistent
sudo apt install iptables-persistent
sudo netfilter-persistent save

# LAN clients need to use this server as their gateway:
# Gateway: <this server's LAN IP>  (e.g., 192.168.1.100)
```

8.3 Monitoring & Bandwidth

Bandwidth Monitoring

```
# Monitor real-time bandwidth on USB tether interface
sudo apt install nload
nload usb0

# Or use iftop for per-connection breakdown
sudo apt install iftop
sudo iftop -i usb0

# Check total data used (useful for limited mobile data):
sudo apt install vnstat
sudo vnstat -i usb0
sudo vnstat -i usb0 --live

# One-liner to see current transfer rates:
watch -n 1 'cat /proc/net/dev | grep usb'
```

8.4 Making Modules Load at Boot

Module Auto-Load at Boot

```
# If rndis_host module doesn't load automatically at boot:

# Add to /etc/modules
echo 'rndis_host' | sudo tee -a /etc/modules
echo 'cdc_ether'  | sudo tee -a /etc/modules

# For iPhone:
echo 'cdc_ncm'    | sudo tee -a /etc/modules
echo 'ipheth'     | sudo tee -a /etc/modules

# Alternative: modprobe config
sudo nano /etc/modprobe.d/usb-tethering.conf
```

```
# Add:
# options rndis_host

# Update initramfs to include modules
sudo update-initramfs -u

# Verify modules are loaded after reboot
lsmod | grep -E 'rndis|cdc|ipheth'
```

9. Quick Reference Cheat Sheet

Scenario	Command
Detect new interface after plug-in	dmesg | tail -20
List all interfaces	ip link show
Bring interface up	sudo ip link set usb0 up
Get IP via DHCP	sudo dhclient usb0
Check IP address	ip addr show usb0
Check routing	ip route show
Add default route	sudo ip route add default via 192.168.42.129 dev usb0
Test internet (IP)	ping -c 3 8.8.8.8
Test internet (DNS)	ping -c 3 google.com
Release DHCP lease	sudo dhclient -r usb0
Renew DHCP lease	sudo dhclient usb0
Remove interface config	sudo ip addr flush dev usb0
Monitor bandwidth	nload usb0
Check NetworkManager	nmcli device status
Restart NetworkManager	sudo systemctl restart NetworkManager
Live kernel USB events	udevadm monitor --subsystem-match=net
USB power management off	echo on | sudo tee /sys/bus/usb/devices/1-1/power/control
Load RNDIS module	sudo modprobe rndis_host
Load iPhone module	sudo modprobe cdc_ncm
Check module loaded	lsmod | grep rndis

10. Summary

USB tethering is one of the simplest and most reliable ways to get internet on a Linux server without Wi-Fi. Here is a condensed flow of what to do:

9. Connect phone → Enable USB tethering on phone
10. Run: dmesg | tail -20 to see the new interface name
11. Run: sudo ip link set usb0 up
12. Run: sudo dhclient usb0
13. Run: ping -c 3 8.8.8.8 to verify
14. Optionally set up udev rules or NetworkManager for auto-connect

TIP For a permanent headless server setup, the udev + helper script method (Section 5.1) combined with systemd-networkd (Section 3.4) is the most robust approach. It will auto-connect whenever the phone is plugged in and tethering is enabled, even after reboots.

With USB tethering configured, you can combine it with Cloudflare Tunnel (Section 8.1) to maintain full remote SSH access from anywhere in the world using just a smartphone data plan — no ISP, no router, no static IP required.

— End of Document —